ACTIVITY 1

S.NO	FIELDS	DESCRIPTION	DATA TYPE	LENGTH CONSTRAINTS	EXAMPLE
1	User Id	The user must use unique identifier	INT	6-8	0405
2	User Name	The user must enter valid name	VARCHAR	10-30	Thenmozhi12
3	Password	Encrypted Password	VARCHAR	8-16	$thenmozhi @123
4	Email	Valid mail	VARCHAR	100	Thenmozhi@ gmail.com
5	Mobile Number	Valid registered number	INT	10	12345667888
6	Gender	Gender of the user	VARCHAR	12	Female
7	Train Number	Official train number	INT	5	12345
8	Train Name	Name of the train	VARCHAR	60	Karnataka Express
9	Total seats	Total available seats	INT	5	1000
10	Booking ID	Everyone should get unique booking ID	INT	10	10001
11	PNR Number	Passenger Name Record Number	INT	15	87268636876787
12	Journey Date	Date of Travel	DATE		26.2.26
13	Quota	General/Tatkal	VARCHAR	15	Taktal
14	Total Fare	Total Ticket cost	DECIMAL	6,2	1990.85
15	Booking status	Confirmed/RAC/waiting	VARCHAR	20	RAC
16	Payment method	UPI/Card/NetBAnking	VARCHAR	50	UPI
17	Refund amount	If the user cancels	DECIMAL	6,2	1650.55
18	Refunf status	Proceed/Pending	VARCHAR	16	Processed
19	Seat Number	Allocated Seat	VARCHAR	10	B2-45
20	Berth Type	The berth type	VARCHAR	16	Lower

ACTIVITY 2

WATERFALL MODEL

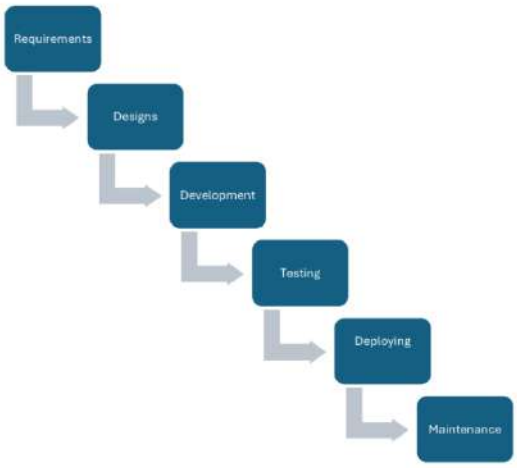
The Waterfall Model is a traditional software development methodology that follows a linear, phase by phase approach, where each phase must be completed before moving

Refund amount	If the user cancels	DECIMAL	6,2	1650.55
Refunf status	Proceed/Pending	VARCHAR	16	Processed
Seat Number	Allocated Seat	VARCHAR	10	B2-45
Berth Type	The berth type	VARCHAR	16	Lower

ACTIVITY 2

WATERFALL MODEL

The Waterfall Model is a traditional software development methodology that follows a linear, phase-by-phase approach, where each phase must be completed before moving to the next. It does not allow backtracking and permits only minimal changes once a phase is completed. The model is simple, well-structured, and easy to manage, offering high predictability and clearly defined milestones throughout the development process.



1. **Requirement Gathering**
Collect all requirements from clients and document them in the **SRS**.

2. **System Design**
Create system architecture, database design, UI design, and overall system flow.

3. **Implementation**
Developers write code module by module based on design documents.

4. **Testing**
Test the completed software for bugs to ensure it meets requirements.

5. **Deployment**
Release the software to users or production environment.

6. **Maintenance**
Fix post-deployment issues, updates, and improvements.

Advantages

1. Easy to Understand:
The model is straightforward and simple, making it easy to learn and follow.
2. Sequential Processing:
Each phase is completed one at a time, ensuring an organized development flow.
3. Well-Defined Stages:
Every phase has clear objectives and deliverables, reducing confusion.
4. Clear Milestones:
Progress is easy to track due to clearly defined milestones.
5. Strong Documentation:
All processes, actions, and results are thoroughly documented.
6. Encourages Best Practices:
Promotes disciplined practices such as define before design and design before *coding*.
7. Suitable for Small and Stable Projects:
Works well for smaller projects where requirements are clearly understood and unlikely to change.

Disadvantages

1 **No flexibility**
Once a phase is completed, it is very difficult to go back and make changes. If requirements change later, it becomes costly and time-consuming.

2 **Late testing**
Testing happens only after the development phase is completed. If major issues are found, fixing them can be difficult.

3 **High risk for unclear requirements**
If the requirements are not clearly understood at the beginning, the final product may not meet customer expectations.

4 **No working software until the end**
The customer cannot see or use a working product until the project is almost finished, which may lead to dissatisfaction.

5 **Not suitable for complex projects**
Waterfall works well only when requirements are simple and fixed. It is not ideal for large or complex projects

AGILE MODEL

AG 4 of 13

The Agile Model was primarily designed to help a project adapt quickly to change requests. So, the main aim of the Agile model is to facilitate quick project completion. To accomplish this task, it's important that agility is required. Agility is achieved by fitting the process to the project and removing activities that may not be essential for a specific project.
Also, anything that is a waste of time and effort is avoided. The Agile Model refers to a group of development processes. These processes share some basic characteristics but do have certain subtle differences among themselves.

Steps in the Agile Model

The Agile Model is a combination of iterative and incremental process models. The phases involve in **Agile (SDLC)Model** are:

1 **Requirement Gathering**
Requirements collected in small user stories; priorities decided with customer.

2 Design
Create simple and flexible design only for the current sprint.

3 Construction / Iteration
Develop features in short cycles (2–4 weeks) and deliver small working software.

4 Testing
Testing happens within each sprint; bugs fixed immediately.

5 Deployment
Working software delivered to users after every sprint.

6 Feedback
Customer feedback used to improve next iterations.

Advantages of Agile Model

1. Flexible to requirement changes
2. Continuous customer involvement
3. Early and frequent delivery of working software

4. Better quality due to continuous testing
5. Reduced project risk

4. e to continuous testing
5. risk
6. Improved team collaboration

Disadvantages of Agile Model

1. Less documentation compared to Waterfall
2. Requires high customer involvement
3. Not suitable for small teams without experience
4. Scope may increase due to frequent changes
5. Difficult to predict final cost and timeline

V MODEL

The V-Model is a sequential software development approach where each development phase has a corresponding testing phase, forming a "V" shape. It is also known as the Verification and Validation Model:

- Verification (left side): Ensures the product is being built correctly.
- Validation (right side): Ensures the right product is being built.

It extends the Waterfall model by planning testing activities in parallel with development. The V-Model is widely used in safety-critical and regulated industries (medical, automotive, aerospace, defence) due to its strict traceability and documentation.

Verification Phases (Left Side)

- Business Requirements Analysis: Defines customer needs; drives planning for User Acceptance Testing (UAT).
- System Design: Establishes architecture; defines System Testing strategy.
- High-Level Design (HLD): Breaks system into modules; planning for Integration Testing.
- Low-Level Design (LLD): Details internal logic of modules; planning for Unit Testing.
- Implementation: Coding phase where designs are converted into actual software.

Validation Phases (Right Side)

- Unit Testing: Verifies individual components against LLD.
- Integration Testing: Ensures modules work correctly together.

- System Testing: Validates complete system against architectural and functional requirements.

ing: Validates complete system against architectural and functional s.

- User Acceptance Testing (UAT): Confirms the product meets business needs and real-world usage expectations.

How Both Sides Connect

Each development phase directly maps to a corresponding test phase, enabling:

- Complete requirement-to-test traceability
- Early test planning and preparation
- Clear linkage between defects and their originating requirements or design gaps\

ACTIVITY 3

Test Execution process what is smoke,regression & sanity testing

Test Execution is the process of executing the tests written by the tester to verify that the developed code, functions, or modules provide the expected result as per the client's or business's requirements. Test Execution comes under one of the phases of the Software Testing Life Cycle (STLC).

Objectives:

1. Test execution ensures that the project runs smoothly and efficiently.
2. It also helps to ensure the application's competency in the global market.
3. Test executions make sure that the requirements are collected correctly and incorporated correctly in design and architecture.
4. It also checks whether the software application meets the requirements or not

Process of Test Execution

The test Execution technique consists of three different phases, which will be carried out to process the test result and ensure the correctness of the required results. In each phase, various activities or work will be carried out by various team members. The three main phases of test execution are the creation of test cases, test case execution, and validation of test results

1.Define Test Objective

In this phase, the tester understands what needs to be tested in the application. The requirements are studied clearly to know the purpose of testing. It helps to decide the

1.Define Test Objective

In this phase, the tester understands what needs to be tested in the application. The requirements are studied clearly to know the purpose of testing. It helps to decide the goal and scope of testing.

2. Test Case Design

In this phase, the tester writes test cases based on the requirements. Test cases include steps to execute, test data, and expected results. This ensures all features are properly covered.

3. Test Execution

In this phase, the tester runs the test cases on the application. The actual result is observed and compared with the expected result. The test case is marked as pass or fail.

4. Result Comparison

Here, the tester checks whether the actual output matches the expected output. If both are the same, the test case passes. If there is a difference, it fails.

5. Defect Reporting

If any test case fails, the tester reports the defect to the development team. The defect details include the issue description, steps to reproduce, and screenshots.

6. Re-Test (if necessary)

After the developer fixes the defect, the tester tests it again to make sure the issue is resolved properly.

7. Test Case Conclusion

At the end, the tester prepares a summary report of testing. It includes total test cases executed, passed, failed, and defect status. This completes the test execution process.

SMOKE TESTING

Smoke Testing is a software testing method that determines whether the employed build is stable or not. It acts as a confirmation of whether the quality assurance team can proceed with further testing. Smoke tests are a minimum set of tests run on each build. Smoke testing is a process where the software build is deployed to a quality assurance environment and verified to ensure the application's stability. Smoke Testing is also known as Confidence Testing or Build Verification Testing.

SANITY TESTING

It is a subset of regression testing. Sanity testing is performed to ensure that the code changes that are made are working properly. Sanity testing is a stoppage to check whether testing for the build can proceed or not. The focus of the team during the sanity testing process is to validate the functionality of the application and not detailed testing. Sanity testing is generally performed on a build where the production deployment is required immediately like a critical bug fix.

SANITY TESTING PROCESS

ON | EVALUATION | TESTING

Identification: Identification is the initial phase in the sanity testing process, where we look for newly added features and components as well as code modifications made to address bugs.
Evaluation: After the identification phase, we will examine recently added features and components and make necessary modifications to ensure they functd and in accordance with the specifications.
Testing: Upon the successful completion of the identification and evaluation phases, we will proceed to the testing phase. In this stage, we examine and evaluate each of the associated parameters, elements, and critical components of the previously analyzed attributed and adjusted them to ensure optimal functionality.

REGRESSION TESTING

Regression Testing involves re-executing a previously created test suite to verify that recent code changes haven't caused new issues. This verifies that updates, bug fixes, or enhancements do not break the functionality of the application

PROCESS OF REGRESSION TESTING

Identify Code Changes: Analyze the source code to determine which areas have been modified, such as new features, bug fixes, or optimizations.
Debug and Fix Failures: If existing test cases fail due to changes, debug the code to identify and resolve defects.
Modify Code: Apply necessary updates to the code to incorporate changes or fixes.
Select Test Cases: Choose relevant test cases from the existing test suite that cover modified and affected areas. Add new test cases if needed to address new functionality.
Execute Regression Tests: Run the selected test cases, either manually or using automated tools, to verify system behavior.
Analyze Results: Review test outcomes to identify regressions, document issues, and recommend fixes.
Retest as Needed: If defects are found, fix them and re-run tests to confirm resolution.

ACTIVITY 4

WHITE BOX/ GREY BOX/ BLACK BOX TESTING

WHITE BOX TESTING

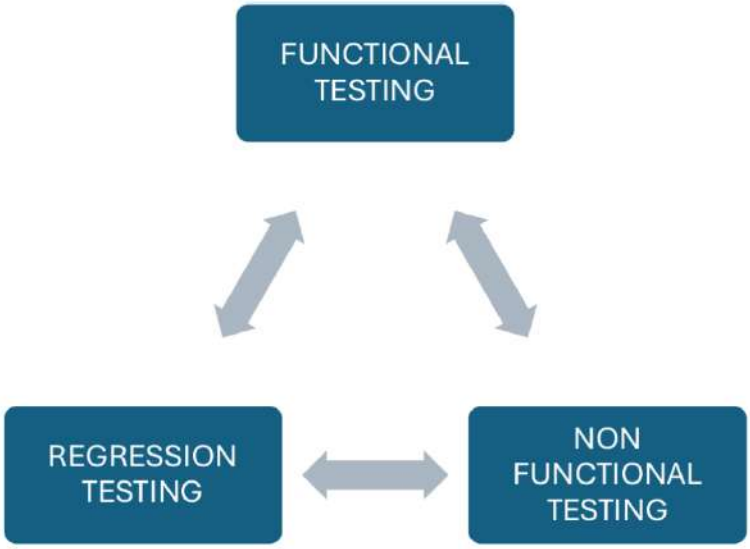
White box testing is a Software Testing Technique that involves testing the internal structure and workings of a Software Application.

- The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.
- White box testing is also known as Structural Testing or Code-based Testing, and it is used to test the software's internal logic, flow, and structure.

- The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.
- White box testing is also known as Structural Testing or Code-based Testing, and it is used to test the software's internal logic, flow, and structure.
- The tester creates test cases to examine the code paths and logic flows to ensure they meet the specified requirements.
- Tools commonly used for whitebox testing include
- **JUnit**: A widely used testing framework for Java applications.
- **NUnit**: A testing framework for .NET applications.
- **TestNG**: A testing framework inspired by JUnit, designed for test configuration and parallel execution.
- **Cucumber**: A tool for behaviour-driven development (BDD) that allows writing tests in plain language.
- **SonarQube**: A tool for continuous inspection of code quality and security vulnerabilities.

BLACK BOX TESTING

Black-box testing is a Type of Software Testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements.



Functional Testing:
Functional testing is a type of black box testing where the system is tested based on

fun[illegible]nts and specifications. In this testing, the tester checks whether each feature of the application works as expected according to the requirement document. The focus is on inputs and outputs without knowing the internal code. It ensures that the system performs the correct functions, such as login, registration, search, and other user operations.

Regression Testing:

Regression testing is performed to make sure that recent changes, bug fixes, or updates in the software have not affected the existing functionality of the application. After modifying the code, testers re-test the already working features to confirm that they still work properly. The main goal of regression testing is to maintain system stability after updates or enhancements.

Non-Functional Testing (NFT):

Non-functional testing focuses on testing the non-functional aspects of the system such as performance, speed, security, usability, reliability, and scalability. Instead of checking what the system does, it checks how well the system performs. For example, it verifies how fast the application loads, how it behaves under heavy traffic, and whether user data is secure

Tools Used :

- **Selenium**: An open-source tool for automating web applications for testing purposes.
- **QTP/UFT (Quick Test Professional/Unified Functional Testing)**: A commercial tool for functional and regression testing.
- **LoadRunner**: A performance testing tool that simulates virtual users to test application performance.
- **Postman**: A tool for testing APIs by sending requests and validating responses.
- **TestComplete**: An automated UI testing tool for desktop, web, and mobile applications.

GREY BOX TESTING

Gray Box Testing is a software testing technique that combines elements of the Black Box Testing technique and the White Box Testing technique. Gray box testing is also known as translucent testing or API testing.

Objectives of Gray Box Testing:

1. To provide the combined advantages of both black box testing and white box testing.
2. To combine the input of developers as well as testers.
3. To improve overall product quality.

4. To reduce the overhead of the long process of functional and non-functional

overhead of the long process of functional and non-functional

5. To provide enough free time to developers to fix defects.
6. To test from the user's point of view rather than a designer's point of view.

Tools used :

- **Burp Suite**: A security testing tool for web applications that allows for both black box and grey box testing.
- **OWASP ZAP (Zed Attack Proxy)**: An open-source tool for finding vulnerabilities in web applications.
- **JMeter**: A performance testing tool that can be used for both functional and load testing, with some knowledge of the internal workings of the application.
- **SoapUI**: A tool for testing web services and APIs, allowing for both functional and security testing.

ACTIVITY 5

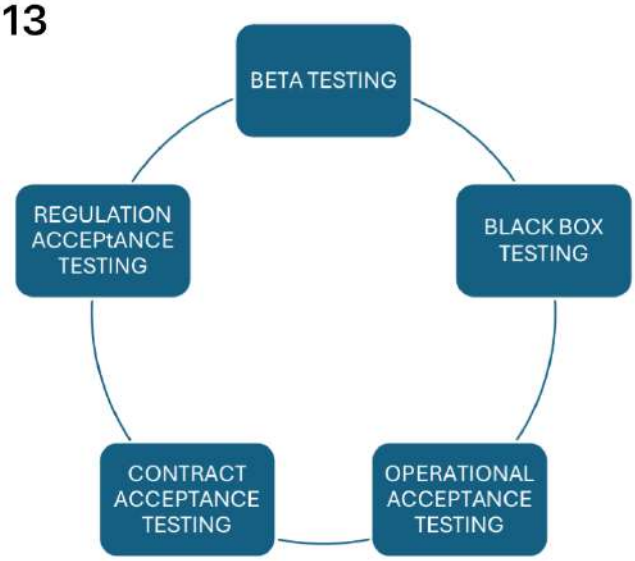
UAT-USER ACCEPTANCE TESTING

User Acceptance Testing (UAT) serves the purpose of ensuring that the software meets the business requirements and is ready for deployment by validating its functionality in a real-world environment. It allows end-users to test the software to ensure it meets their needs and operates as expected, helping to identify and fix any issues before the final release. UAT is crucial for quality assurance and customer satisfaction, as it ensures that the software is user-friendly, reliable, and meets all specified criteria.

What is the purpose of UAT?

The purpose of User Acceptance Testing (UAT) is to identify bugs in software, systems, and networks that may cause problems for users. UAT ensures that software can handle real-world tasks and perform to development specifications. Users are allowed to interact with the software before its official release to see if any features were overlooked or if any bugs exist.

TYPES OF USER ACCEPTANCE TESTING



1. **Beta Testing**
   Beta testing is done by real users before the final release of the product. The software is given to a limited group of external users to check how it works in a real environment. They provide feedback about bugs, usability issues, and improvements.
   2. Black Box Testing
   In black box testing, users test the software without knowing the internal code or structure. They only check whether the system works correctly based on inputs and expected outputs. The main focus is on functionality.
   3. Operational Acceptance Testing (OAT)
   Operational Acceptance Testing checks whether the system is ready for real-time business operations. It verifies backup, recovery, security, performance, and system stability before going live.
   4. Contract Acceptance Testing
   Contract acceptance testing ensures that the software meets all requirements mentioned in the contract between the client and the development team. The system is tested to confirm that agreed features are delivered.
   5. Regulation Acceptance Testing
   Regulation acceptance testing checks whether the software follows legal rules, industry standards, and government regulations. It ensures the system complies with required policies before release

SYSTEM AND INTEGRATION TESTING

System Testing:

While developing a software or application product, it is tested at the final stage as a

While developing a software or application product, it is tested at the final stage as a whole by combining all the product modules and this is called as System Testing. The primary aim of conducting this test is that it must fulfill the customer/user requirement specification.

It is also called an end-to-end test, as is performed at the end of the development. This testing does not depend on system implementation; in simple words, the system tester doesn't know which technique between procedural and object-oriented is implemented.

This testing is classified into **functional** and **non-functional** requirements of the system. In functional testing, the testing is similar to black-box testing which is based on specifications instead of code and syntax of the programming language used. On the other hand, non-functional testing, checks for performance and reliability by generating test cases in the corresponding programming language.

Integration Testing:

This testing is the collection of the modules of the software, where the relationship and the interfaces between the different components are also tested. It needs coordination between the project-level activities of integrating the constituent components at a time. The integration and integration testing must adhere to a building plan for the defined integration and identification of the bug in the early stages. However, an integrator or integration tester must have programming knowledge, unlike a system tester